# AI-ASSISTED CLAIMS TRIAGE SYSTEM

Business Requirements Document

Author: Aravind Kumar Thupilly
Version: 1.0
Date: 03/06/2025
Document ID: 01

# Table of Contents

# 1. Introduction

## 1.1 Purpose of This Document

This document describes, in a single place:

- The **business requirements** for AI-assisted claims triage and fraud detection.
- The **technical design** implemented in Python, SQL, and MySQL.
- The **data model (ERD)** and how it supports fraud rules and dashboards.

## 1.2 Intended Audience

- Business stakeholders (Operations, Fraud / SIU, Management)
- Data / BI Analysts
- Data Engineers & Developers
- Auditors / Compliance teams

## 1.3 Project Overview

The system ingests health insurance claims and patient-related data, cleans and enriches it using a Python **AI Triage Model**, loads the data into MySQL, and then applies **six business rules** to identify:

1. Duplicate claims
2. High-risk patients using multiple providers
3. Provider overcharging (amount above capped amount)
4. Duplicate services (same procedure repeated)
5. Sudden daily spikes in risk scores
6. Claims with high fraud risk scores

Alerts are stored in real_time_alerts and visualized through dashboards (e.g., high-risk patients, risk score trends, duplicate claims by month, fraud alerts).

## 1.4 Scope

In scope:

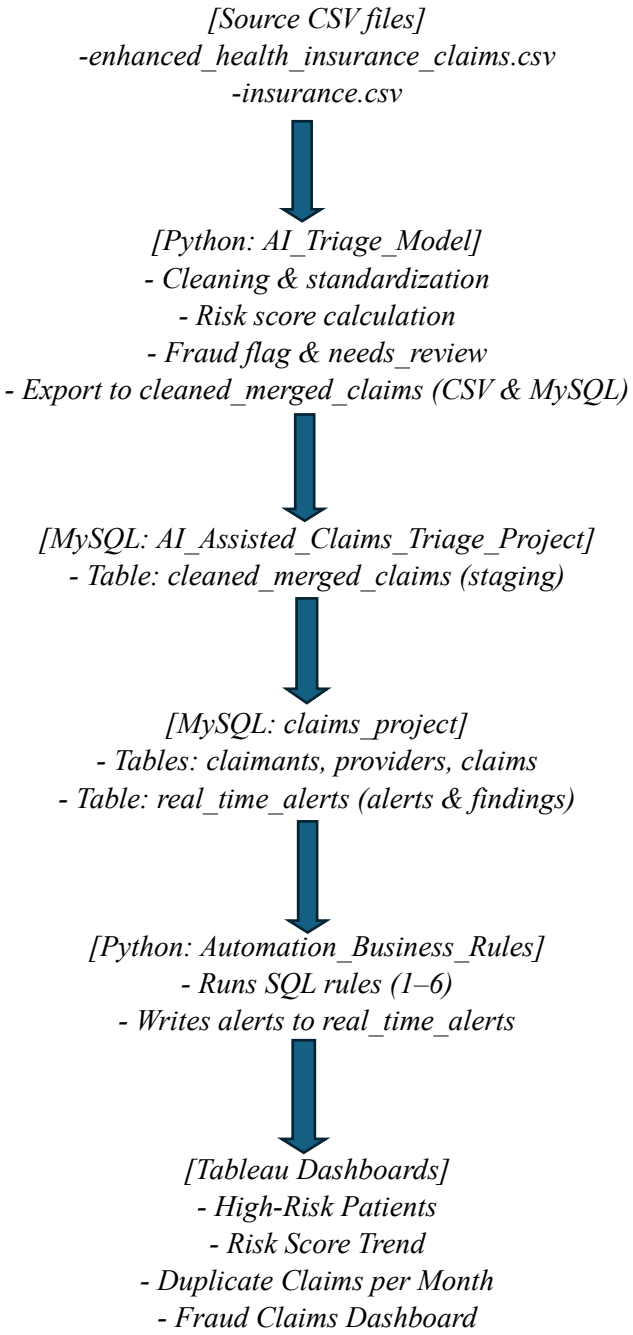
- Data preparation and risk scoring (AI_Triage_Model.ipynb).
- Relational schema design & ERD (claims, claimants, providers, real_time_alerts).
- Implementation of 6 fraud / risk rules in SQL and Python.
- Automation of alert generation and persistence.
- Testing using mock data and unit tests.

Out of scope:

- Real-time integration with claim core systems for auto-denial.
- Advanced ML model training for prediction beyond current rule-based scoring.

# 2. End-to-End Solution Overview

## 2.1 High-Level Architecture




**2.2 Data Flow Summary**

1. **Data Preparation (Python):**
   - Reads raw claims & insurance datasets.
   - Cleans data, fills missing values, caps outliers, computes risk scores, and flags.
2. **Staging (MySQL – cleaned_merged_claims):**
   - Stores cleaned and enriched claim-level data.
3. **Core Schema (claims_project):**
   - Normalizes data into claimants, providers, claims tables.
4. **Business Rules Execution:**
   - Six rules implemented in SQL and Python: duplicate, overcharging, risk spikes, etc.
5. **Alerts (real_time_alerts):**
   - Stores rule outputs as structured alert records, linked back to claims via claim_id.
6. **Dashboards:**
   - Visualize high-risk patients, trends, duplicates, and alerts.

## 3. Data Model & Entity-Relationship Diagram (ERD)

### 3.1 Staging Layer: cleaned_merged_claims

Purpose: store cleaned, enriched claim data before it is normalized into the operational schema.

Key fields:

- claim_id (identifier)
- age, bmi, smoker, region, smoker_flag
- procedure_code, amount, amount_capped
- status, claim_type
- claim_date, claim_month
- needs_review, fraud_flag
- timestamp, risk_score

This table is partitioned by year of claim_date to support performance.

### 3.2 Core Operational Schema: claims_project

Tables (simplified):

1. **claimants** – demographics and behaviour
    - claimant_id (PK)
    - age, bmi, smoker, region, smoker_flag
2. **providers** – claim-submitting entities
    - provider_id (PK)
3. **claims** – core fact table
    - claim_id (PK)
    - claimant_id (FK → claimants)
    - provider_id (FK → providers)
    - procedure_code, amount, amount_capped, status, claim_type, claim_date, claim_month
    - needs_review, fraud_flag, timestamp, risk_score
4. **real_time_alerts** – alerts + findings
    - alert_id (PK)
    - claim_id, original_claim, duplicate_claim
    - rule_name, rule_triggered, alert_status, alert_level
    - risk_score, alert_score
    - provider_id, claimant_id
    - detection_time, detection_timestamp, triggered_on
    - provider_fraud_history, procedure_code, amount, days_apart, needs_review

### 3.3 ERD Visual

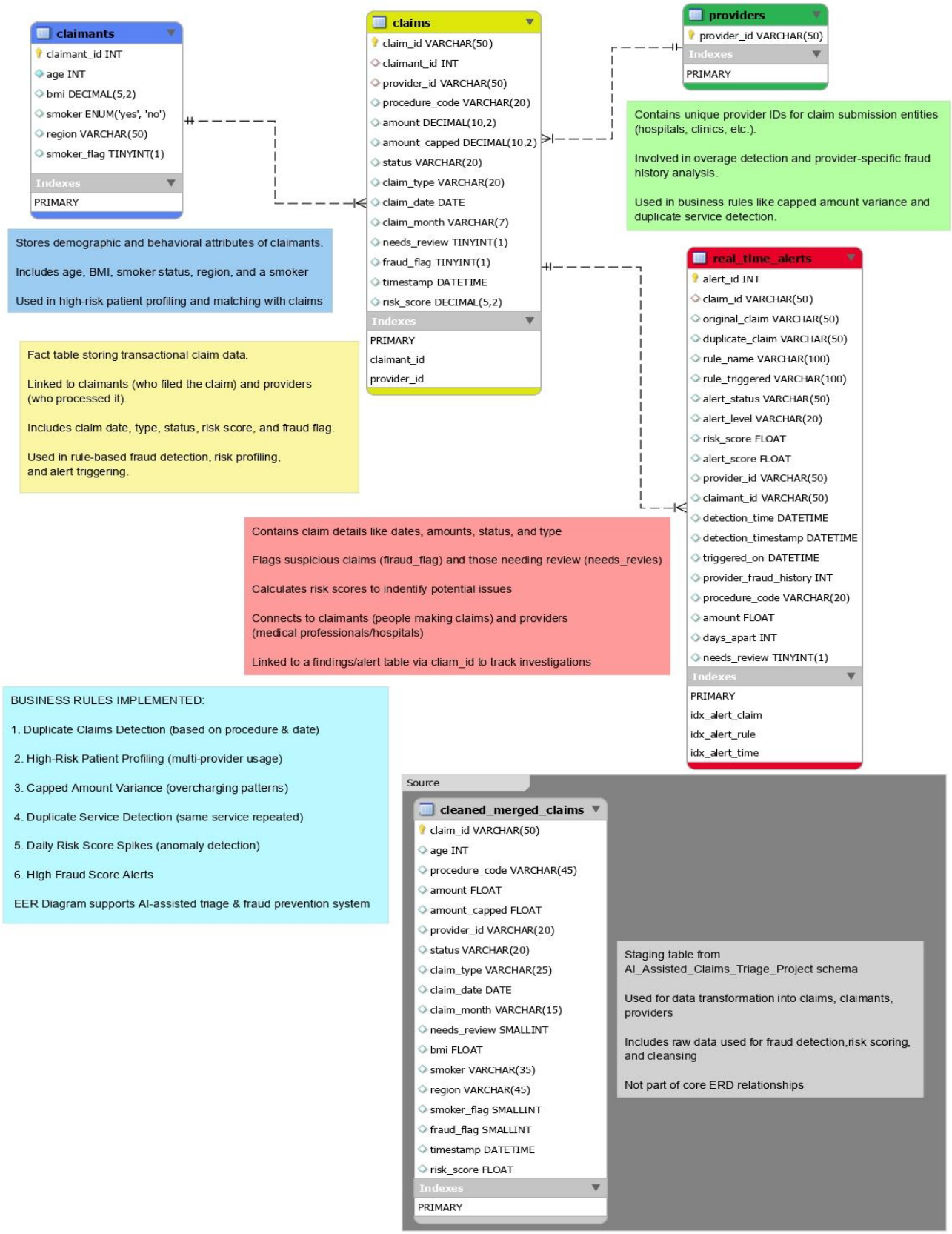



**Figure 1: Logical Data Model.**

## 4. Data Preparation & AI Triage Model (Python)

This section documents how the AI Triage Model cleans and prepares data before loading it into MySQL.

### 4.1 Source Data Sets

- enhanced_health_insurance_claims.csv – raw claims (amount, date, patient, provider, procedure).
- insurance.csv – patient-level attributes (smoker, BMI, region, etc.).

### 4.2 Data Cleaning & Standardization

**Python Codes:**

```
# Load raw data
claims = pd.read_csv("enhanced_health_insurance_claims.csv")
insurance = pd.read_csv("insurance.csv")

# Rename columns for consistency
claims = claims.rename(columns={
    "PatientAge": "age",
    "ClaimAmount": "amount",
    "ClaimDate": "claim_date",
    "ClaimID": "claim_id",
    "ClaimType": "claim_type",
    "ClaimStatus": "status",
    "ProcedureCode": "procedure_code",
    "ProviderID": "provider_id"
})
```

**Standardization:**

- Convert **smoker**, **status**, **claim_type** to consistent casing.
- Fill missing region with "**Unknown**", missing **smoker** with "**No**", and missing bmi with median.

### 4.3 Feature Engineering & Risk Scoring

Key steps:

- Cap outliers in amount at the **99th percentile** and store in **amount_capped**.
- Create binary flags:
  - smoker_flag from smoker
  - fraud_flag based on high amounts **(top 5%)**
  - needs_review when fraud_flag = **1** or risk_score **> 75**
- Compute risk_score using age, BMI, and smoking status, scaled to **0–100**.

**Python Codes:**

```
merged["risk_score"] = (
    0.3 * merged["age"] / 100 +            # normalized age
    0.5 * (merged["bmi"] - 10) / 40 +      # normalized BMI
    0.2 * merged["smoker_flag"]
) * 100
```

### 4.4 Export to Staging Table & MySQL

- Remove duplicate claim_id rows, keep **4,500 final records**.
- Export to CSV **cleaned_merged_claims.csv.**
- Load into MySQL **AI_Assisted_Claims_Triage_Project.cleaned_merged_claims** table.

This staging table becomes the **single source of truth** for subsequent transformations into **claims_project**.

## 5. SQL Schema, Indexing & Performance Design

### 5.1 Database & Table Creation

Two main databases:

- AI_Assisted_Claims_Triage_Project – staging (cleaned_merged_claims).
- claims_project – normalized schema and alerts.

**MySQL Queries:**

```
CREATE DATABASE AI_Assisted_Claims_Triage_Project;
USE AI_Assisted_Claims_Triage_Project;

CREATE DATABASE claims_project;
USE claims_project;

CREATE TABLE claimants (
   claimant_id INT AUTO_INCREMENT PRIMARY KEY,
   age INT NOT NULL,
   bmi DECIMAL(5,2),
   smoker ENUM('yes', 'no'),
   region VARCHAR(50),
   smoker_flag BOOLEAN
);

CREATE TABLE providers (provider_id VARCHAR(50) PRIMARY KEY);

CREATE TABLE claims (
   claim_id VARCHAR(50) PRIMARY KEY,
   claimant_id INT, provider_id VARCHAR(50),
   procedure_code VARCHAR(20), amount DECIMAL(10,2),
   amount_capped DECIMAL(10,2), status VARCHAR (20),
   claim_type VARCHAR(20), claim_date DATE,
   claim_month VARCHAR(7), needs_review BOOLEAN,
   fraud_flag BOOLEAN, timestamp DATETIME,
   risk_score DECIMAL(5,2),
   FOREIGN KEY (claimant_id) REFERENCES claimants(claimant_id),
   FOREIGN KEY (provider_id) REFERENCES providers(provider_id)
);
```

**real_time_alerts** table is also created with indexes for performance.

### 5.2 Indexing Strategy

Example:

- ***idx_fraud*** on cleaned_merged_claims(fraud_flag)
- ***idx_procedure*** on procedure_code
- ***idx_date*** on claim_date

Indexes on real_time_alerts:

- ***idx_alert_claim*** on claim_id
- ***idx_alert_rule*** on rule_triggered
- ***idx_alert_time*** on alert_level

**5.3 Partitioning Strategy**

**cleaned_merged_claims** is partitioned by year of claim_date:

```
ALTER TABLE cleaned_merged_claims
PARTITION BY RANGE (YEAR(claim_date)) (
  PARTITION p2023 VALUES LESS THAN (2024),
  PARTITION p2024 VALUES LESS THAN (2025)
);
```

This improves performance for time-based queries and rules.

**6. Fraud & Risk Business Rules: Functional View**

**6.1 Rule Catalogue**

| Rule | Name | Business Purpose |
|---|---|---|
| 1 | Duplicate Claims Detection | Find claims that looks like duplicates for same patient. |
| 2 | High-Risk Patient Profiling | Find patients using **>= 3** providers (doctor shopping). |
| 3 | Capped Amount Variance | Detect providers charging more than the allowed cap. |
| 4 | Duplicate Service Prevention | Detect repeat same-procedure claims within **90 days.** |
| 5 | Sudden Surge in Risk Score | Detect sudden spikes in overall daily claim risk. |
| 6 | High Fraud Score Alerts | Highlight claims with very high risk/fraud flags. |

**6.2 Rule 1: Duplicate Claims Detection**

**Business meaning:** If a patient submits multiple claims with very similar procedures (matching first 3 characters of the procedure code) within **90 days**, those may be duplicates and should be reviewed.

**SQL Implementation:**

```
SELECT
   a.claim_id AS original_claim,
   b.claim_id AS duplicate_claim,
   a.claimant_id, a.procedure_code AS original_procedure,
   a.claim_date AS original_date, b.claim_date AS duplicate_date,
   DATEDIFF(b.claim_date, a.claim_date) AS days_apart
FROM claims a
JOIN claims b ON a.claimant_id = b.claimant_id
 AND a.procedure_code LIKE CONCAT(SUBSTRING(b.procedure_code, 1, 3), '%')
 AND a.claim_id <> b.claim_id
 AND ABS(DATEDIFF(a.claim_date, b.claim_date)) <= 90;
```

This result is fed into Python to calculate a risk score and create High alert records (see Section 7.2)

**6.3 Rule 2: High-Risk Patient Profiling (Multiple Providers)**

**Business meaning:** If a claimant uses **3 or more** different providers, they may be **"doctor shopping"** or coordinating across providers, which is **higher risk.**

**SQL Implementation:**

```
SELECT claimant_id, COUNT(DISTINCT provider_id) AS unique_providers
FROM claims
GROUP BY claimant_id
HAVING unique_providers >= 3;
```

In Python, results are converted to alerts with rule_name = 'Multiple_Providers' and alert_level = 'Medium'.

**6.4 Rule 3: Capped Amount Variance (Provider Overcharging)**

**Business meaning:** If a provider charges more than the capped (allowed) amount consistently, they may be overcharging.

**SQL Implementation:**

```
SELECT provider_id, procedure_code, COUNT(*) AS claim_count,
    ROUND(AVG((amount - amount_capped) / amount_capped * 100), 2) AS avg_overage_pct
FROM claims
WHERE amount > amount_capped
GROUP BY provider_id, procedure_code
HAVING avg_overage_pct > 0
ORDER BY avg_overage_pct DESC;
```

Python wraps this into alerts with rule_name = 'Provider_Overcharging' and alert_level = 'High'.

**6.5 Rule 4: Duplicate Service Prevention (Same Procedure)**

**Business meaning:** If the same patient receives the same procedure again within **1–90 days**, it may be unnecessary or fraudulent repetition.

**SQL Implementation**

```
SELECT a.claimant_id, a.claim_id AS claim1, b.claim_id AS claim2, a.procedure_code,
        a.claim_date AS date1, b.claim_date AS date2, DATEDIFF(b.claim_date, a.claim_date) AS days_apart
FROM claims a
JOIN claims b ON a.claimant_id = b.claimant_id
 AND a.procedure_code = b.procedure_code AND a.claim_id <> b.claim_id
 AND DATEDIFF(b.claim_date, a.claim_date) BETWEEN 1 AND 90;
```

Python labels these as Repeat_Claims_Same_Procedure with medium alert level.

**6.6 Rule 5: Sudden Surge in Risk Score (Daily Spike)**

**Business meaning:** Compare today's average claim risk score with the previous **7 days**. If today's average is **> 120%** of the past week's average, we raise an alert.

**SQL Implementation:**

```
WITH date_info AS (
  SELECT MAX(DATE(claim_date)) AS latest_date,
      DATE_FORMAT(MAX(DATE(claim_date)), '%Y-%m-%d') AS latest_date_formatted,
      DATE_FORMAT(MAX(DATE(claim_date)) - INTERVAL 1 DAY, '%Y-%m-%d') AS yesterday_formatted,
      DATE_FORMAT(MAX(DATE(claim_date)) - INTERVAL 7 DAY, '%Y-%m-%d') AS seven_days_ago_formatted
  FROM claims
)
```

```
SELECT di.latest_date_formatted AS analysis_date,
      (SELECT AVG(risk_score) FROM claims
       WHERE DATE(claim_date) = di.latest_date) AS today_avg,
      (SELECT AVG(risk_score) FROM claims
       WHERE DATE(claim_date) BETWEEN di.seven_days_ago_formatted AND di.yesterday_formatted) AS
prev_7d_avg,
      CASE
       WHEN (SELECT AVG(risk_score) FROM claims WHERE DATE(claim_date) = di.latest_date)
            >
            1.2 * (SELECT AVG(risk_score) FROM claims
                 WHERE DATE(claim_date) BETWEEN di.seven_days_ago_formatted AND di.yesterday_formatted)
       THEN 'ALERT: Risk score spike detected'
       ELSE 'No significant spike detected'
      END AS alert_status
FROM date_info di;
```

Python converts spike results into alerts with rule_name = 'Sudden_Surge_Risk_Score' and alert_level = 'Critical Risk'.

**6.7 Rule 6: High Fraud Score Alerts**

**Business meaning:** Any claim with confirmed fraud **(fraud_flag = 1)** or a **very high-risk score** should be escalated.

**SQL Implementation:**

```
SELECT c.claim_id, c.claimant_id, c.provider_id, c.risk_score,
      CASE
        WHEN c.fraud_flag = 1 THEN 'Confirmed Fraud'
        WHEN c.risk_score > 90 THEN 'Critical Risk'
        WHEN c.risk_score > 75 THEN 'High Risk'
      END AS alert_level,
      COALESCE(p.fraud_history_count, 0) AS provider_fraud_history,
      c.claim_date
FROM claims c
LEFT JOIN (
   SELECT provider_id, COUNT(*) AS fraud_history_count
   FROM claims
   WHERE fraud_flag = 1
   GROUP BY provider_id
) p ON c.provider_id = p.provider_id
WHERE (c.fraud_flag = 1 OR c.risk_score > 75)
  AND c.claim_date BETWEEN '2023-01-01' AND '2024-07-08'
ORDER BY c.fraud_flag DESC, c.risk_score DESC, provider_fraud_history DESC
LIMIT 100;
```

Python labels these as High_Fraud_Score_Alert with alert_level = 'Critical Risk'.

**7. Automation & Real-Time Alerts (Python)**

All Python automation is documented in **Automation_Business_Rules.ipynb**.

**7.1 real_time_alerts Structure**

Alerts use a consistent schema with columns listed in ALERT_COLS, such as claim_id, rule_name, alert_level, alert_score, provider_id, claimant_id, days_apart, needs_review, etc.

**7.2 Python Automation Functions per Rule**

Key functions:

- run_automation(engine) – Rule 1: duplicate claims detection and alert push.
- run_multiple_providers_rule(engine) – Rule 2.
- run_provider_overcharging_rule(engine) – Rule 3.
- run_repeat_claim_rule(engine) – Rule 4.
- run_risk_spike_rule(engine) – Rule 5.
- run_high_fraud_score_alert(engine) – Rule 6.

Each function:

1. Calls the corresponding **SQL detection function** (e.g., detect_high_risk_duplicates, detect_provider_overcharging).
2. Calls a **prepare_alerts** function to add rule metadata and timestamps.
3. Saves alerts safely into **real_time_alerts** via a shared save function (push_to_real_time_alerts, save_alerts_to_sql, etc.).

**7.3 Data Validation & Safe Writes**

- _ensure_valid_claim_ids(engine, df) checks that **claim_id** exists in the **claims** table before writing alerts.
- If no new alerts, functions print "**no alerts to save**" and do not write anything.

**8. Testing & Quality Assurance (Mock Data)**

Testing logic lives in **Testing_with_Mock_Data.ipynb**.

**8.1 Mock Data Testing Strategy**

For each rule, a **small synthetic DataFrame** is created to simulate specific patterns (duplicates, overcharging, etc.). Assertions ensure that each rule detects what it should.

**8.2 Unit Test Functions per Rule**

- test_rule1_duplicate_claims_detection() – verifies Rule 1.
- test_rule2_high_risk_patients() – verifies Rule 2.
- test_rule3_capped_amount_variance() – verifies Rule 3.
- test_rule4_duplicate_service_prevention() – verifies Rule 4.
- test_rule5_risk_score_spike() – verifies Rule 5.
- test_rule6_high_fraud_score() – verifies Rule 6.

**8.3 Automated Test Harness**

run_all_mock_tests() executes all six tests and prints **PASS/FAIL** results:

**Python Code:**

```
#Testing mock data
def run_all_mock_tests():
    test_rule1_duplicate_claims_detection()
    test_rule2_high_risk_patients()
    test_rule3_capped_amount_variance()
    test_rule4_duplicate_service_prevention()
    test_rule5_risk_score_spike()
    test_rule6_high_fraud_score()
    print("\n All mock rule tests passed successfully.")
```

This confirms that all rules work correctly on controlled scenarios before running them on production data.

## 9. Dashboards & Visualizations

### 9.1 High-Risk Patient Profiling View

- X-axis: smoker vs non-smoker
- Y-axis: average age
- Color: count of patients

Business message: "***Smokers and older patients are at higher risk***; we can see this distribution clearly."

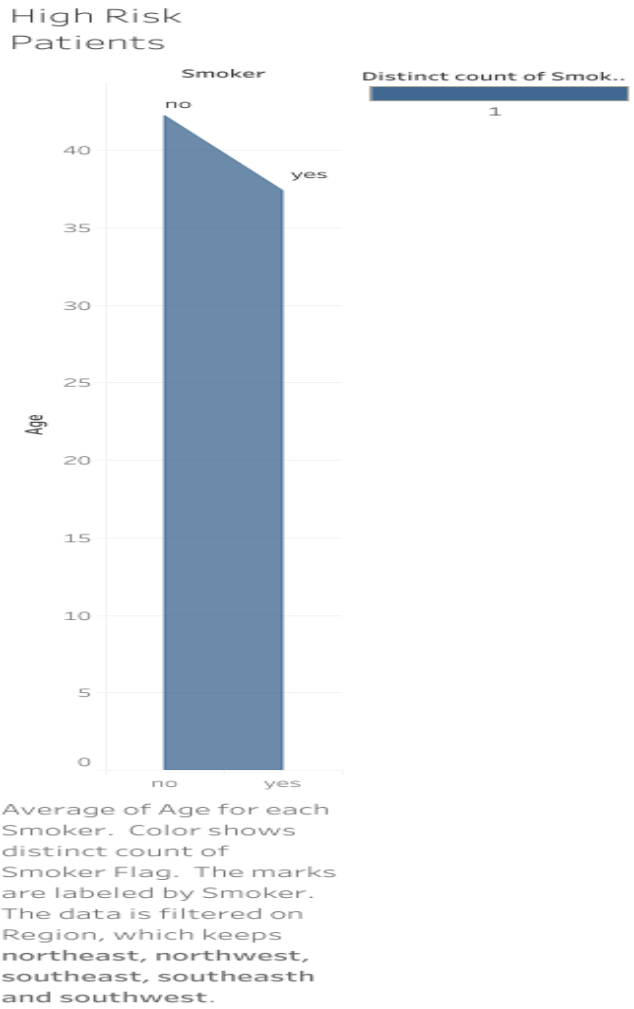



**Figure 2: High Risk Patients**

### 9.2 Risk Score Trend View

- X-axis: Year
- Y-axis: average age
- Shows whether overall risk is rising or falling.

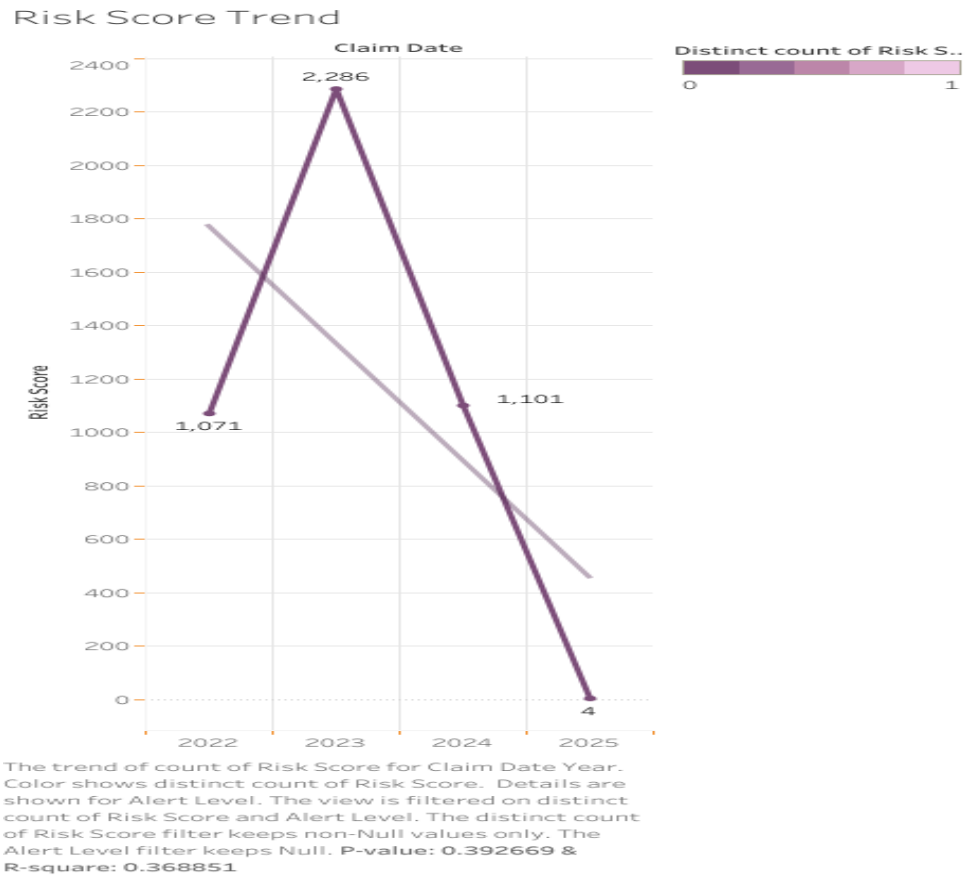



**Figure 3: Risk Score Trend**

### 9.3 Duplicate Claims per Month View

- X-axis: month
- Y-axis: number of duplicate claims

Used to monitor effectiveness of rules and process improvements over time.

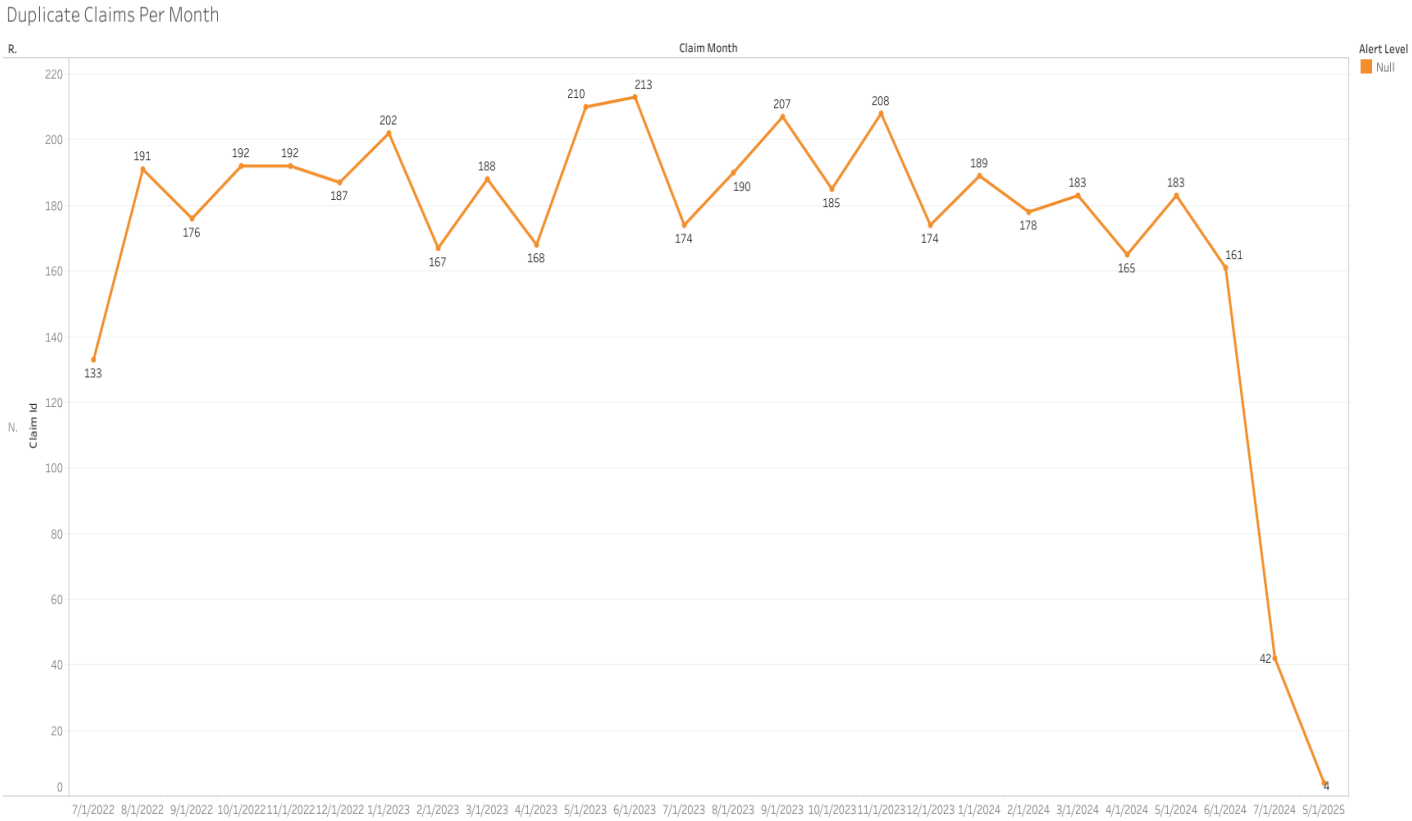



**Figure 4: Duplicate Claims Per Month**

### 9.4 Fraud Claims Dashboard (Combined)

- Combines:
  - High-risk patients
  - Risk score trends
  - Duplicate claims trends
- Filters: region, alert level, rule name

Explain in one paragraph how an analyst would use the dashboard to prioritize cases.

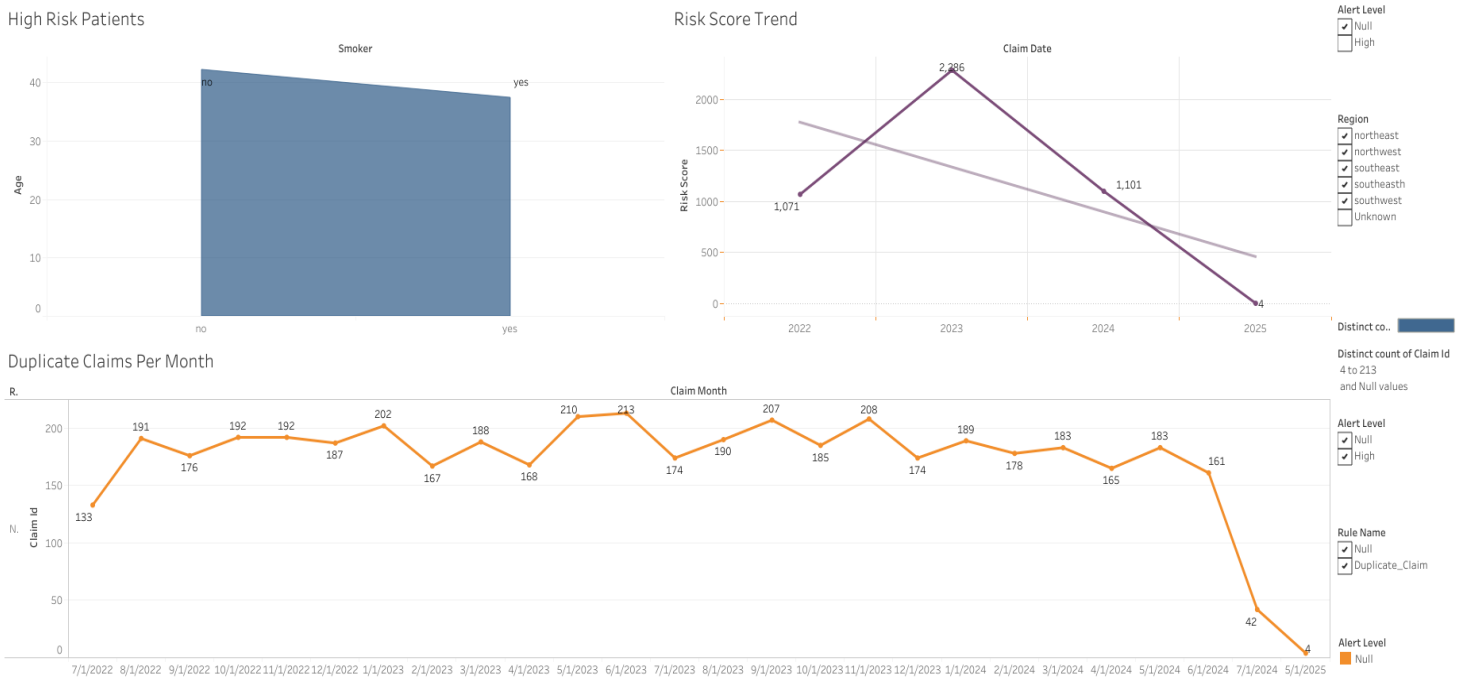



**Figure 5: Fraud Claims Dashboard**

## 10. Glossary of Terms

This glossary is written for non-technical stakeholders. It explains the main terms that appear in the diagrams, rules and dashboards.

- Claim: A request for payment submitted to an insurer for a medical service. One row in the claims table represents one claim.
- Claimant (Patient): The person who filed the claim. Stored in the claimants table with attributes such as age, BMI, smoker status and region.
- Provider: The hospital, clinic or doctor that submitted or delivered the service. Stored in the providers table and referenced from each claim.
- Procedure Code: A code that identifies the medical service performed (for example, PROC123). The first 3 characters are sometimes used to detect similar procedures in duplicate-claim rules.
- Claim Date: The date when the service was performed or the claim was recorded, used for time windows (e.g. 0–90 days) and trend analysis.
- Claim Amount / Capped Amount:
  - Claim amount: the original value requested in the claim.
  - Capped amount: the adjusted amount after applying caps to very high values so that outliers do not distort analysis.
- Risk Score: A numeric value, typically between 0 and 100, that summarises how risky a claim or claimant is. Higher values (for example > 75) indicate elevated risk; very high values (for example > 90) indicate critical risk.
- Fraud Flag: A binary indicator that marks a claim as confirmed or strongly suspected fraud. Often used together with risk score to prioritise alerts.

- Needs Review: A flag that signals the claim should be manually reviewed by an analyst, usually because the risk score is high or a business rule has been triggered.
- Duplicate Claim: A claim that appears to be a repeat or duplicate of another claim for the same claimant, with similar procedures and dates (for example, within 0–90 days).
- High-Risk Patient Profiling: Analysis used to identify patients who show risky patterns, such as visiting 3 or more distinct providers, which may indicate "doctor shopping."
- Capped Amount Variance / Provider Overcharging: The percentage by which a provider's charged amount exceeds the capped amount (for example, 10% or 25% above cap). Used to detect potential systematic overcharging.
- Sudden Surge in Risk Score: A situation where today's average risk score is significantly higher (for example > 120%) than the average over the previous 7 days, which may indicate a new fraud pattern or system issue.
- Alert / Real-Time Alert: A record in the real_time_alerts table that documents why a claim was flagged, by which rule, when, and at what severity level.
- Rule Name / Rule Triggered:
    - Rule name: the label of the rule (e.g. "Duplicate_Claim", "Provider_Overcharging").
    - Rule triggered: a short description of the condition (e.g. "Same procedure within 90 days").
- Alert Level: The severity assigned to an alert, such as Medium, High, or Critical Risk, based on thresholds in the underlying data.
- Alert Score: A numeric score (often 0–100) that indicates the intensity of the alert and helps analysts rank which cases to look at first.
- Staging Table (cleaned_merged_claims): Intermediate table containing cleaned and enriched data from Python before it is split into claims, claimants, and providers.
- ETL (Extract, Transform, Load):
  The process of:
    - Extracting data from source files,
    - Transforming/cleaning it in Python, and
    - Loading it into the MySQL databases for rules and dashboards.
- Index: A database structure that speeds up searches and filters on important columns (for example claim_date, provider_id, risk_score) without scanning the entire table.
- Partitioning: Splitting a large table into physical segments, often by year, so time-based queries run faster and scale better.

**11. Appendices**

These appendices are optional but very useful to demonstrate your technical capability. They show **representative** SQL queries and Python snippets used in the project. In your Word document, you can format them as code blocks or monospaced text.

**11.1 Key SQL Queries**

This appendix provides a high-level view of the most important SQL components.
Full, up-to-date implementations of all rules and DDL scripts are maintained in the project repository:

- **GitHub portfolio:**
  https://github.com/Aravind16-ai
- **Project website:**
  https://aravind16.lovable.app/

**11.1.1 Loading Cleaned Data into claims**

This query loads cleaned and enriched records from the staging table (cleaned_merged_claims) into the main claims table in the claims_project database. It:

- Matches each staged record to an existing claimant based on age, BMI, smoker status, region and smoker flag.
- Preserves risk-related fields such as **risk_score**, **fraud_flag** and **needs_review**.
- Ensures that only valid, deduplicated claims are inserted into the production schema.

**Reference:**
Full insert script available in the **sql_queries.sql** file in the **GitHub** repository.

### 11.1.2 Rule 1: Duplicate Claims Detection

Rule 1 identifies potential duplicate claims for the same claimant where:

- Procedure codes are similar (for example, matching on the first 3 characters), and
- Claim dates are within a defined window (for example **0–90 days**).

The **functional behaviour and thresholds** for this rule are explained in **Section 6.2**.

**Code reference:** The complete SQL implementation for **11.1.2** is available in the SQL scripts in the GitHub repository: https://github.com/Aravind16-ai (See the SQL file or Python notebook corresponding to Rule 1.)

### 11.1.3 Rule 5: Sudden Surge in Risk Score

Rule 5 detects a sudden surge in average risk score by:

- Calculating today's average risk score across all claims.
- Comparing it to the average risk score over the previous **7 days**.
- Raising an alert when today's value exceeds a defined threshold (for example **> 120%** of the prior 7-day average).

The **business logic and interpretation** of this rule are described in **Section 6.6**.

**Code reference:** The full SQL for **11.1.3**, including date-window logic and alert status calculation, is maintained in the GitHub repository: https://github.com/Aravind16-ai

### 11.1.4 Rule 6: High Fraud Score Alert System

Rule 6 flags claims that are highly suspicious based on:

- A confirmed **fraud_flag** on the claim, and/or
- A **risk_score** above defined thresholds (for example **> 75** for High Risk, **> 90** for Critical Risk), and
- Provider fraud history (how many prior fraudulent claims are linked to the same provider).

The **business behaviour, thresholds and ranking logic** for this rule are documented in **Section 6.7**.

**Code reference:** The complete SQL for **11.1.4**, including joins to provider fraud history and date filters, is available in the project repository: https://github.com/Aravind16-ai. Additional context and examples are also presented on the project website: https://aravind16.lovable.app/

## 11.2 Key Python Snippets

This appendix summarises the main Python components used in the project.
The **behaviour and purpose** of each component are described earlier in:

- **Section 4:** Data Preparation & AI Triage Model (Python)
- **Section 7:** Automation & Real-Time Alerts (Python)
- **Section 8:** Testing & Quality Assurance (Mock Data)

The **full, up-to-date Python code** (Jupyter notebooks and scripts) is maintained in the project repositories:

- **GitHub (code reference):**
  https://github.com/Aravind16-ai
- **Project website / portfolio:**
  https://aravind16.lovable.app/

### 11.2.1 AI Triage Model: Data Preparation & Risk Scoring

The AI Triage Model notebook is responsible for:

- Reading raw claims and insurance datasets.
- Cleaning and standardising fields (dates, text values, missing data, outliers).
- Performing feature engineering (capped amounts, smoker flags, derived dates).

- Calculating the claim-level **risk score** on a 0–100 scale.
- Setting **fraud_flag** and **needs_review** based on thresholds (for example scores > 75 or very high amounts).
- Exporting the curated dataset into the **cleaned_merged_claims** table in MySQL.

The detailed logic flows and examples are described in **Section 4**.

**Code reference:** The complete AI Triage implementation is available as a Jupyter notebook in ***AI_Triage_Model.ipynb*** in the GitHub repository: https://github.com/Aravind16-ai. Additional explanation and visuals are presented on the project website: https://aravind16.lovable.app/

### 11.2.2 Automation of Business Rules & Alert Writing

The automation notebook coordinates the execution of all fraud and risk rules and writes alerts into the real_time_alerts table. It typically includes:

- A **main orchestration function** that connects to the database and runs each rule in turn.
- Dedicated functions such as:
  - "duplicate claims" rule runner (Rule 1),
  - "multiple providers / high-risk patients" rule runner (Rule 2),
  - "provider overcharging" rule runner (Rule 3),
  - "repeat service within 90 days" rule runner (Rule 4),
  - "risk score spike" rule runner (Rule 5),
  - "high fraud score alerts" rule runner (Rule 6).
- Helper functions to **enrich** rule outputs with metadata:
  - rule_name, rule_triggered, alert_level, alert_score, timestamps.
- A shared **save routine** that validates claim IDs and then inserts alerts into real_time_alerts in a safe, consistent way.

The business behaviour and thresholds for each rule are documented in **Section 6** and the overall automation flow in **Section 7**.

**Code reference:** The complete automation logic often named something like ***Automation_Business_Rules.ipynb*** is available in the GitHub repository: https://github.com/Aravind16-ai. A narrative description of how automation works with screenshots is also available on: https://aravind16.lovable.app/

### 11.2.3 Testing with Mock Data & Unit Test Harness

The testing notebook focuses on verifying that each rule behaves as expected using controlled, synthetic datasets. It includes:

- **Mock data generators** that create small Data Frames designed to trigger or not trigger each rule (for example, deliberate duplicates, overcharging cases, repeated procedures within 90 days, spike patterns in risk scores).
- **Unit test functions** for each rule, such as:
  - Test for duplicate claims detection (Rule 1).
  - Test for high-risk patients using multiple providers (Rule 2).
  - Test for capped amount variance / provider overcharging (Rule 3).
  - Test for duplicate service within the time window (Rule 4).
  - Test for sudden surge in risk score (Rule 5).
  - Test for high fraud score conditions (Rule 6).
- A **combined test runner** that executes all tests and prints a clear pass/fail summary, ensuring changes to the rules do not break existing logic.

The overall testing strategy and objectives are described in **Section 8**.

**Code reference:** The full mock testing and unit test harness in ***Testing_with_Mock_Data.ipynb*** is available in the GitHub repository: https://github.com/Aravind16-ai. Testing highlights and key results can also be showcased on the project website: https://aravind16.lovable.app/.